Funktionen

- Analog zu Methoden in Java, kann man Funktionen definieren
  `function <name>([<parameterliste>]): <typ> { <Anweisungen> }`
- Beispiel:

```
function double (n : number) : number {
    return n * 2;
}
```

- Aufruf:
  Eine Funktion wird über den Namen aufgerufen und die Argumente werden übergeben.
  Beispiel:
  `console.log(double(21));`

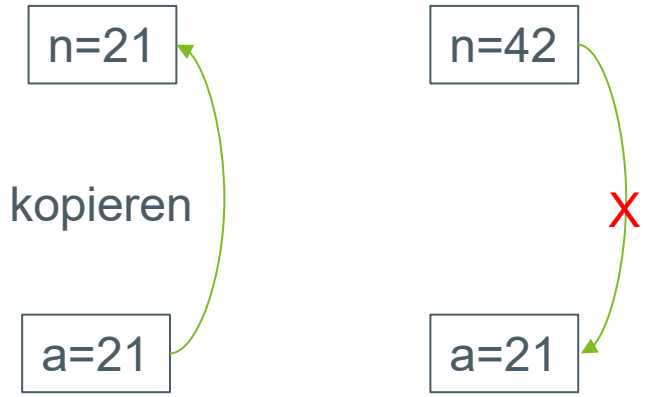
Parameterübergabe – call-by-value

- Primitive Datentypen (number, string, boolean (auch, wenn Sie Teil einer Union sind!) werden immer „per-Wert“ übergeben, indem der Wert des Argumentes in den entsprechenden Parameter kopiert wird.

- Beispiel

```
function double (n : number) : number {
    n = n * 2;
    return n;
}

let a : number = 21;
console.log("double(a)= " + double(a));
console.log("a= " + a);
```



- Ausgabe?

Parameterübergabe call-by-reference

- Alle nicht-primitiven Typen werden „per-Referenz“ übergeben
  - Statt eines Wertes wird die Adresse des Argumentes im Speicher übergeben
  - Parameter und Argument liegen also an der gleichen Stelle im Speicher!
- Beispiel

```
function doubleList(list: number[]): void {
    list[0] = list[0] * 2;
    list[1] = list[1] + 2;
}
let list: number[] = [5, 2];
console.log(list);
doubleList(list);
console.log(list);
```

- Ausgabe?
- Ist das Programm korrekt? Wie geht das besser?

Auflösung

```
function doubleList(list: number[]): void {
    list[0] = list[0] * 2;
    list[1] = list[1] + 2;
    return list[0] + list[1];
}
let list: number[] = [5, 2];
console.log(list);
doubleList(list);
console.log(list);
```



```
function doubleList(list: number[]): number[] {
    let list2 : number[] = [];
    list2[0] = list[0] * 2;
    list2[1] = list[1] + 2;
    return list2;
}
let liste: number[] = [5, 2];
console.log(liste);
let neueListe = doubleList(liste);
console.log("Alte Liste: " + liste
            + ", neue Liste: „
            + neueListe);
```

Versuchen Sie Seiteneffekte zu vermeiden, da Sie nur durch Analyse der Funktion verstehen, was diese tut.

Best Practice: Die Funktion sollte nur das tun, was in der Deklaration steht!

Default Parameter und „Rest-Parameter“

- In der Parameterliste kann man auch Default-Werte angeben, so dass man beim Aufruf die entsprechenden Argumente weglassen kann:

```
function test(a:number=0, b:number):number {
return a+b;
}
console.log(test(1));
console.log(test(1,2));
```

- Mittels … kann man eine Parameterliste variabler Länge angeben:

```
function beliebig(...numbers) {
    return numbers.length;
}
console.log(beliebig(1, 2, 4));
```

- Der Rest-Parameter muss immer der letzte Parameter sein. Warum?

Rekursive Funktionen

- Eine Funktion heißt rekursiv, wenn sie sich selbst wieder aufruft.
- Oft sind mathematische Funktionen rekursiv definiert
- Beispiel:

$$n! = \begin{cases} 1, & ,falls\ n = 1 \\ n * (n-1)! & ,sonst \end{cases}$$

```
function factorial (n : number) : number {
    if (n===1) return 1;
    else return n * factorial(n-1);
}

for (i=1; i<=10;i++){
    console.log("factorial(" + i + ")= " + factorial(i));
}
```

Gültigkeitsbereiche (Scope)

- Jede Variable ist nur in ihrem Gültigkeitsbereich sichtbar
- In jedem Block, einem Bereich, der durch { und } eingefasst ist, können Variablen deklariert werden.
- „Innere“ Variablen können „äußere“ Variablen verdecken.
- Variablen, die nicht in einem Block deklariert sind, sind global (und müssen dort deklariert werden).
- Beispiel

```
{ // hier beginnt ein Block
    let x = 0

    { // hier beginnt ein zweiter Block
        let x=1
        let y=2
        console.log(x);
    }
    console.log(x);
    console.log(y);
}
```

Ausgabe: 1

Ausgabe: 0

Fehler! y ist nicht deklariert im äußeren Block!

Funktionstypen

- Beispiel

```
function inc (n : number):number {
    return n+1;
}
```

- Was ist der Datentyp von *inc*? Hat *inc* einen Typen?
- Ja! Der Typ von *inc* ist `(n : number) => number`

- Und man kann diese Funktionen einer Variablen zuweisen:

```
let incFct : (n:number) => number = inc;
```

- Und aufrufen:

```
console.log(incFct(3));
```

Lambda-Funktionen (Arrow Functions)

- Oft ist es gar nicht nötig, einer Funktion einen Namen zu geben. Dann verwendet man eine sogenannte Lambda-Funktion.
- Beispiel

```
let paul: (a: number, b: number) => number = (a, b) => { return a + b; };
console.log(paul(3,4));
```

- Schreiben Sie eine Lambda-Funktion, die von zwei Zahlen die größere zurückgibt.

```
const maxx = (a:number, b:number): number => {return a>b ? a : b; }
```

Klassen

- Eine **Klasse** fasst zusammengehörige Variablen und Funktionen („Methoden“) zusammen.
- Klassen sind selber wieder Datentypen und Variablen von einem solchen Typ heißen **Objekte** oder Instanzen der Klasse.
- Beispiel

```
class Person {
    vorname  : string;
    nachname : string;
    alter    : number;
    constructor(vorname : string, nachname : string){
        this.vorname = vorname;
        this.nachname = nachname;
    }
    name():string {
        return this.vorname + " " + this.nachname;
    }
}

let otto = new Person("Otto", "Schultz");
console.log(otto.name());
```

Instanzvariablen

Konstruktor

Methode

Anlegen eines Objektes

Aufruf einer Methode

Vererbung

- Eine Klasse kann eine andere erweitern („extends“) und erbt somit alle Eigenschaften dieser Klasse.
- Beispiel

```
class Student extends Person {
    matrikelnr : number;
    constructor(vorname: string, nachname: string, matrikelnr : number) {
        super(vorname, nachname);
        this.matrikelnr = matrikelnr;
    }
    name():string {
        return super.name()+" (" + this.matrikelnr + ")";
    }
}

let alice = new Student("Alice", "Wonderland", 567890);
console.log(alice.name());
```

Student ist Unterklasse, Person Oberklasse

Aufruf des Konstruktors der Oberklasse

Sichtbarkeit für Klassen („visibility“)

- Für Attribute und Methoden können Sichtbarkeitsregeln angegeben werden
  - `public`: auf solche Attribute kann überall zugegriffen werden (“`public`” ist default)
  - `protected`: auf solche Attribute dürfen nur abgeleitete Klassen zugreifen
  - `private`: auf solche Attribute darf nur die Klasse selbst zugreifen.
- Beispiel

```
class Person1 {
   protected vorname    : string;
   private    nachname : string;
…
}
class Student1 extends Person1 {
    private matrikelnr : number;
    …
    name():string {
        return this.vorname + " "
             + this.nachname;
    }
}
```

ok!?

ok!?

Statische Klassenvariablen

- Durch das Schlüsselwort static können in einer Klasse definierte Variablen als Klassenvariablen ausgezeichnet werden.
- Klassenvariablen existieren einmal pro Klasse, nicht pro Instanz!
- Zugriff mittels <Klassenname>.<Variablenname> !
- Beispiel:

```
class Static {
    public static scounter : number = 0;
    public icounter : number;

    constructor() {
        this.icounter = 0;
        Static.scounter++;
    }
}

let s1 : Static = new Static();
console.log("scounter: \t" + Static.scounter + "\t icounter: \t" + s1.icounter);
let s2 : Static = new Static();
console.log("scounter: \t" + Static.scounter + "\t icounter: \t" + s2.icounter);
```

Klassenvariable

Instanzvariable

Ausgabe?

```
scounter: 1   icounter:    0
scounter: 2   icounter:    0
```

Abstrakte Klassen

- Eine abstrakte Klasse ist eine Klasse, die nicht selbst instanziiert werden kann
- Abstrakte Klassen bieten eine Möglichkeit, gemeinsame Eigenschaften und Methoden zu definieren, die von konkreten (nicht-abstrakten) Unterklassen geerbt werden können.
- Abstrakte Klassen können abstrakte Methoden haben, die in der abstrakten Klasse deklariert werden, aber in den abgeleiteten Klassen implementiert werden müssen.

```
abstract class Form {
  posX : number;
  posY:number;

   constructor(posX: number, posY: number) {
      this.posX = posX;
      this.posY = posY;
   }
   abstract flaeche() : number;
}
```

```
class Rechteck extends Form {
    laenge : number;
    breite : number;

    constructor(posX: number, posY: number, laenge: number,
                breite: number) {
        super(posX, posY);
        this.laenge = laenge;
        this.breite = breite;
    }

    flaeche(): number { return this.breite * this.laenge; }
}
```